



Milestone2

CSE480: Machine Vision

Name	ID	Sec.
Zeyad samer lotfy	2100820	3
Mohamed Islam salah	2101439	3
Moaz Gamal Alsayed	2101231	1
Mohamed Montasser	2100543	1
Fatma Samy Ahmed	2100660	2

Provided to :

DR / Hossam Eldin Hassan

Eng/ Dina Zakaria

Overview and Objective

Milestone 2 focuses on system integration, optimizer comparison, advanced evaluation, and performance analysis of deep learning models for human behaviour understanding, this milestone concentrates on:

- 1-Designing a robust facial emotion recognition model using a CNN architecture.
- 2-Performing systematic optimizer comparison under identical experimental conditions.
- 3-Appling regularization, learning rate scheduling, and fine-tuning strategies.
- 4-Conducting detailed quantitative evaluation using confusion matrices, classification reports, and per-class accuracy.

Dataset and Preprocessing (Emotion Recognition)

1. Dataset Description

The FER-2013 dataset is used for facial emotion recognition. The dataset contains grayscale facial images categorized into the following seven emotion classes:

Angry, Disgust, Fear, Happy, Sad, Surprise, Neutral

The dataset is already split into training and test folders, which are directly loaded by the implemented pipeline.

2. Image Preprocessing Pipeline

Each image passes through the following preprocessing steps exactly as implemented in the code:

Grayscale Loading

Images are loaded using OpenCV in grayscale mode to match the FER-2013 format.

Resizing

All images are resized to **48 × 48 pixels**, which is the fixed input size for the CNN model.

CLAHE Enhancement

Contrast Limited Adaptive Histogram Equalization (CLAHE) is applied to enhance facial details and improve robustness to illumination variations.

Normalization

Pixel values are normalized to the range **[0, 1]**.

Dimensional Expansion

A channel dimension is added, resulting in an input shape of:

(48, 48, 1)

3. Data Augmentation

To improve generalization and reduce overfitting, real-time data augmentation is applied only during training using ImageDataGenerator. The implemented augmentations include:

Rotation ($\pm 15^\circ$)

Width and height shifting ($\pm 15\%$)

Shearing

Zooming

Horizontal flipping

Validation and test sets are not augmented, ensuring fair evaluation.

Model Architecture (Emotion Recognition CNN)

1. Design Rationale

A **custom VGG-style convolutional neural network** is implemented from scratch. The architecture is intentionally deep to capture fine-grained facial features while maintaining training stability through batch normalization and dropout.

2. Network Structure

The implemented model consists of the following components:

Convolutional Blocks

Block 1: Two convolution layers with 64 filters, followed by ReLU activation, batch normalization, and max pooling.

Block 2: Two convolution layers with 128 filters, followed by ReLU activation, batch normalization, and max pooling.

Block 3: Two convolution layers with 256 filters, followed by ReLU activation, batch normalization, and max pooling.

Block 4: Two convolution layers with 512 filters, followed by ReLU activation, batch normalization, and global average pooling.

Classifier

Fully connected layer with 1024 units + ReLU

Dropout (0.5)

Fully connected layer with 512 units + ReLU

Dropout (0.5)

Output softmax layer with 7 emotion classes

This architecture is implemented exactly as defined in the `build_small_vgg()` function.

Training Strategy and Optimizer Comparison

1.Optimizers Used

Two optimization algorithms are implemented and compared:

Stochastic Gradient Descent (SGD) with momentum and Nesterov acceleration

Adam Optimizer

Each optimizer is trained from scratch using identical datasets, augmentation settings, and evaluation metrics.

2. Training Configuration

Batch size: 128

Maximum epochs: 50

Loss function: Categorical Cross-Entropy

Evaluation metric: Accuracy

3.Callbacks and Regularization

The following callbacks are implemented:

`ReduceLROnPlateau`

Automatically reduces the learning rate when validation accuracy stops improving.

`ModelCheckpoint`

Saves the best model based on validation accuracy for each optimizer.

Batch normalization and dropout layers are used throughout the network to reduce overfitting.

Experimental Results and Evaluation

1. Optimizer Performance Comparison

For each optimizer, the following metrics are recorded directly from training logs:

Best validation accuracy

Final validation accuracy

Total training time

Validation accuracy and loss curves are plotted to visually compare convergence behaviour and stability.

2. Test Set Evaluation

The best-performing optimizer (based on validation accuracy) is selected, and its corresponding model is evaluated on the unseen test set.

The evaluation includes:

Test accuracy

Test loss

Confusion matrix (normalized)

Classification report (precision, recall, F1-score)

Per-class accuracy

All metrics are computed using sklearn and TensorFlow evaluation functions exactly as implemented in the code.

3. Fine-Tuning Phase

A fine-tuning step is performed on the best saved model:

Optimizer: SGD

Learning rate: $1e-4$

Additional epochs: 20

The fine-tuned model is saved separately and re-evaluated on the test set. This step resulted in improved generalization and higher test accuracy compared to the non-fine-tuned model.

Visualization and Analysis Tools

The following visual analysis tools are implemented and used:

Training vs. validation accuracy curves

Training vs. validation loss curves

Optimizer comparison plots

Normalized confusion matrices

Sample test image predictions with true and predicted labels

These visualizations provide insight into learning dynamics, overfitting behavior, and class-wise performance.

Discussion and Observations

Based on the implemented experiments:

Adam optimizer converges faster during early training stages.

SGD with momentum demonstrates improved stability during fine-tuning.

Data augmentation and CLAHE preprocessing significantly enhance robustness.

Certain emotion classes (e.g., happy and surprise) achieve higher accuracy due to more distinctive facial features.

All observations are derived directly from the generated evaluation metrics and plots.

Appendix: Code Reference

All results and discussions in this report are based solely on the implemented Python code, including model architecture definitions, training scripts, evaluation functions, and visualization utilities provided in the project notebook.